

RabbitMQ DLQ Demonstration Guide

Notification Service API · Docker Compose · separate WorkerService · RabbitMQ Management UI

Goal. Demonstrate that a poison notification command is retried three times, then preserved in `nsa.notifications.bulk.dlq` instead of being lost or retried forever.

Architecture being demonstrated

```
POST /api/v2/notifications/bulk → API saves job → RabbitMQ main queue → Worker retry × 3 → dead-letter exchange → DLQ
```

The API and WorkerService run separately. RabbitMQ keeps the command durable outside the API process. A final failure is rejected without requeue and routed to the Dead-Letter Queue (DLQ).

Part 1 — Prepare the local environment

1 Open PowerShell in the repository root

```
Set-Location 'C:\Users\Full Scale\projects\Projects'
Get-Location
```

Why: all paths in the commands below are relative to this folder.

2 Start Docker Desktop and verify it

```
docker version
docker compose version
```

Why: the Week 3 stack uses Linux SQL Server and RabbitMQ images. Docker Desktop must be running in Linux-container mode before Compose can start the database, broker, API, and worker.

3 Create and complete the local `.env` file

```
Copy-Item notification-api/.env.example notification-api/.env
notepad notification-api/.env
```

Replace every blank value. For a local-only demo, use your own strong values or this working example:

```
WEEK3_SQL_PASSWORD=LocalSql!2026Demo
WEEK3_RABBITMQ_USERNAME=nsa_demo
WEEK3_RABBITMQ_PASSWORD=LocalRabbit!2026Demo
```

Setting	Why it is needed
WEEK3_SQL_PASSWORD	Creates/authenticates SQL Server's <code>sa</code> login for API and worker access.
WEEK3_RABBITMQ_USERNAME	Creates the broker user used by the API publisher, worker consumer, and Management UI.
WEEK3_RABBITMQ_PASSWORD	Authenticates that broker user everywhere it is needed.

Security: `notification-api/.env` is local-only and ignored by Git. Do not commit it or display it while screen-sharing after passwords are added.

4 Validate before starting containers

```
docker compose `
  --env-file notification-api/.env `
  -f notification-api/compose.week3.yml `
  config --quiet
```

Expected result: no output and exit code 0.

If Compose reports a required variable is missing: the file was copied but its values are still blank. Reopen `notification-api/.env`, set all three values from Step 3, save it, and validate again. Compose stops deliberately because it cannot create the broker user or authenticate the application without them.

5 Start and verify the stack

```
docker compose `
  --env-file notification-api/.env `
  -f notification-api/compose.week3.yml `
  up --build --detach

docker compose `
  --env-file notification-api/.env `
  -f notification-api/compose.week3.yml `
  ps
```

Expected result: SQL Server and RabbitMQ become healthy; API and worker start after dependencies are ready.

6 Sign in to RabbitMQ Management

1. Open `http://localhost:15672`.
2. Use `WEEK3_RABBITMQ_USERNAME` and `WEEK3_RABBITMQ_PASSWORD` from your `.env` file.
3. Open **Queues and Streams**.

Why: this UI is the visible evidence that the broker owns the failed message. Confirm `nsa.notifications.bulk.v1` and `nsa.notifications.bulk.dlq` exist.

Part 2 — Demonstrate retry and DLQ routing

7 Enable the local poison-message hook and recreate the worker

Run these commands in the same PowerShell window. The variable is session-only; if you close the window, set it again before recreating the worker.

```
$env:WEEK3_FAILURE_INJECTION_SUBJECT = '[week3-poison]'  
  
docker compose `--env-file notification-api/.env `--f notification-api/compose.week3.yml `up --detach --force-recreate worker
```

Why: the worker receives `RabbitMq:FailureInjectionSubject` and deliberately fails only a pending item whose subject exactly equals `[week3-poison]`.

8 Open the worker-log terminal

Open a **second** PowerShell window at the repository root. Run:

```
Set-Location 'C:\Users\Full Scale\projects\Projects'  
  
docker compose `--env-file notification-api/.env `--f notification-api/compose.week3.yml `logs --follow --tail 20 worker
```

Every line beginning with `worker-1 |` is a worker log. Keep this window open while you submit the poison job. `Ctrl+C` stops log-following only; it does not stop the worker container.

9 Submit a new poison job

Open `http://localhost:8080/swagger`. Execute `POST /api/v2/notifications/bulk` with one valid item whose subject is exactly `[week3-poison]`:

```
{  
  "notifications": [  
    {  
      "recipientEmail": "demo@example.test",  
      "channel": 1,  
      "subject": "[week3-poison]",  
      "body": "Controlled local DLQ demonstration."  
    }  
  ]  
}
```

Record the returned job ID and correlation ID. The API should return `202 Accepted`.

10 Show the third attempt and DLQ

Attempt	Expected worker action
1	Intentional failure; command is republished with retry count 1 .
2	Intentional failure; command is republished with retry count 2 .
3	Job becomes <code>DeadLettered</code> ; worker rejects without requeue.

Refresh RabbitMQ Management → **Queues and Streams** → `nsa.notifications.bulk.dlq` . Show the ready message. In Swagger, call `GET /api/v2/notifications/bulk/{jobId}` and show status `DeadLettered` .

What to say: “The command exhausted its bounded retry budget. It was rejected without requeue and RabbitMQ routed it through the dead-letter exchange to a durable DLQ for investigation and controlled replay.”

Troubleshooting: no third attempt or dead-letter message

If you see `unknown flag: --force-` **and then** `recreate` **is not recognized:** the option `--force-recreate` was split across two lines. Docker did not recreate the worker, so it did not receive the poison-message setting.

Fix: use the exact multi-line command in Step 7. A PowerShell continuation character (```) must be the final character on each continued line; do not add spaces after it.

If the log says `Acknowledged command ...` : that job was processed normally. Confirm the hook was enabled and submit a *new* job with the exact subject `[week3-poison]` .

To show relevant recent worker evidence after the test:

```
docker compose `
  --env-file notification-api/.env `
-f notification-api/compose.week3.yml `
logs --tail 200 worker |
Select-String -Pattern 'Dead-lettering|retry|poison'
```

Expected result: after three attempts, a line reports that the command is being dead-lettered. Then the DLQ contains the retained message.

11 Disable the local test hook

```
Remove-Item Env:WEEK3_FAILURE_INJECTION_SUBJECT -ErrorAction SilentlyContinue

docker compose `
  --env-file notification-api/.env `
-f notification-api/compose.week3.yml `
up --detach --force-recreate worker
```

Do this after the demonstration so normal local jobs are no longer deliberately failed.

Evidence checklist

- Compose status shows API, worker, SQL Server, and RabbitMQ running.
- Worker log shows bounded retry and dead-lettering.
- RabbitMQ Management shows a ready message in `nsa.notifications.bulk.dlq` .
- Job-status endpoint shows `DeadLettered` .

Scope: this demonstrates durable queueing, bounded retries, and DLQ routing. It is at-least-once delivery; Outbox, Inbox/deduplication, and automated replay are later reliability work.

DLQ Demonstration Guide Addendum

Demonstrating automatic recovery of a rejected notification command

Purpose. This addendum supplements the original DLQ demonstration. It proves that a rejected command can be recovered safely: first force a job into the DLQ, then let the dedicated `dlq-recovery-worker` reset and replay it, and finally verify the admin and visitor notifications were created.

What this demonstration proves

1. Main worker fails the poison command three times.
2. SQL job becomes DeadLettered; RabbitMQ places the command in `nsa.notifications.bulk.dlq`.
3. Recovery worker validates that RabbitMQ marked it as rejected from the main queue.
4. Recovery worker saves `RecoveryPending`, clears `x-retry-count`, and confirmed-publishes to recovery queue.
5. Recovery queue delays five seconds, then routes to main queue.
6. Main worker processes the pending admin and visitor items; job becomes Completed.

Prerequisites

1. Docker Desktop is running.
2. `notification-api/.env` exists with the local SQL Server and RabbitMQ credentials.
3. Start the current five-service stack from the repository root:

```
docker compose --env-file notification-api/.env \
  -f notification-api/compose.week3.yml up --build --detach

docker compose --env-file notification-api/.env \
  -f notification-api/compose.week3.yml ps
```

Wait until `api`, `worker`, and `dlq-recovery-worker` are healthy.

Step 1 — Pause automatic recovery and enable the controlled poison failure

Stopping the recovery worker lets you inspect the rejected command in the DLQ before it is replayed. The poison subject is a local-only demonstration hook; it is not normal application behavior.

```
$compose = 'notification-api/compose.week3.yml'

docker compose --env-file notification-api/.env -f $compose stop dlq-recovery-worker

$env:WEEK3_FAILURE_INJECTION_SUBJECT = '[week3-poison]'
```

```
docker compose --env-file notification-api/.env -f $compose up --detach --force-recreate worker
Remove-Item Env:WEEK3_FAILURE_INJECTION_SUBJECT
```

Important: remove the environment variable after Compose recreates the primary worker. The worker container retains the value used at creation; removing it from your PowerShell session prevents accidentally applying it to the later normal-worker restart.

Step 2 — Submit a two-recipient bulk job

Use unique emails if you repeat the guide. This sample creates one admin and one visitor notification item.

```
$run = [Guid]::NewGuid().ToString('N')
$admin = "dlq-admin-$run@example.test"
$visitor = "dlq-visitor-$run@example.test"

$payload = @{ notifications = @(
    @{ recipientEmail = $admin; channel = 4; subject = '[week3-poison]'; body = 'Recovered admin notification.' },
    @{ recipientEmail = $visitor; channel = 4; subject = '[week3-poison]'; body = 'Recovered visitor notification.' }
) } | ConvertTo-Json -Depth 5

$job = Invoke-RestMethod -Method Post `
    -Uri 'http://127.0.0.1:8080/api/v2/notifications/bulk' `
    -ContentType 'application/json' -Body $payload

$job.jobId
```

Step 3 — Confirm the job reached the DLQ

Poll the status endpoint until status is `DeadLettered`. The worker has now exhausted its three primary processing attempts and rejected the command without requeue.

```
$jobId = $job.jobId
do {
    Start-Sleep -Seconds 2
    $status = Invoke-RestMethod -Uri "http://127.0.0.1:8080/api/v2/notifications/bulk/$jobId"
    $status
} while ($status.status -ne 'DeadLettered')
```

Optional RabbitMQ Management inspection: open `http://localhost:15672`, then inspect `Queues` and `Streams` → `nsa.notifications.bulk.dlq`. The command remains there because the recovery worker is paused.

Step 4 — Restore normal processing and start the recovery worker

```
# Recreate the primary worker with failure injection disabled.
docker compose --env-file notification-api/.env -f $compose up --detach --force-recreate worker

# Start the dedicated DLQ recovery worker.
docker compose --env-file notification-api/.env -f $compose start dlq-recovery-worker

# Follow both logs while recovery happens.
docker compose --env-file notification-api/.env -f $compose logs --follow worker dlq-recovery-worker
```


Expected log sequence:

- Marked dead-lettered bulk notification job ... as pending recovery.
- Replayed dead-lettered notification command ... replay 1 of 3.
- Acknowledged command ... from the primary worker.

Step 5 — Verify the recovered job and both notifications

```
do {
  Start-Sleep -Seconds 2
  $status = Invoke-RestMethod -Uri "http://127.0.0.1:8080/api/v2/notifications/bulk/$jobId"
  $status
} while ($status.status -ne 'Completed')

Invoke-RestMethod -Uri ("http://127.0.0.1:8080/api/v2/notifications?recipientEmail=" +
[uri]::EscapeDataString($admin))
Invoke-RestMethod -Uri ("http://127.0.0.1:8080/api/v2/notifications?recipientEmail=" +
[uri]::EscapeDataString($visitor))
```

Success criteria. The job reports `Completed`, `processedCount: 2`, `succeededCount: 2`, and each recipient query returns exactly one notification with subject `[week3-poison]`.

Safety behavior to demonstrate or explain

Situation	Expected result
Malformed body, wrong command type, unknown job, or a message not rejected from the main queue	The recovery worker publish-confirms it to <code>nsa.notifications.bulk.parking</code> ; it is retained for inspection, not discarded or looped.
The same replay fails repeatedly	After <code>RabbitMq:MaxDeadLetterReplayAttempts</code> (default 3), it is parked with reason <code>replay-limit-exhausted</code> .
SQL or recovery publication is temporarily unavailable	The recovery worker NACKs with <code>requeue</code> , retaining the original DLQ delivery until it can safely complete the state change and confirmed publication.

Finish

```
docker compose --env-file notification-api/.env -f $compose ps
Invoke-WebRequest -UseBasicParsing http://127.0.0.1:8080/health/ready
```

All five Compose services should be healthy, and the API readiness endpoint should return `200 Healthy`.

Appendix added 14 August 2026. It documents the `notification.dlq.worker` automatic replay path introduced after the original guide.

DLQ Demonstration Guide — Refactor Update

Use this addendum with the existing walkthrough. It reflects the new shared worker host and focused delivery handlers; the externally visible queue topology and demonstration outcome are unchanged.

Scope. This is a local Compose demonstration only. `RabbitMq__FailureInjectionSubject` is an intentional test hook. Never enable it in a production deployment.

1. Start the refactored stack

Run these commands from the repository root. The worker Dockerfiles now include `notification.worker.shared`, so `--build` validates the actual deployable dependency graph.

```
docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml up --build --detach

docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml ps
```

Wait for all five services to report healthy: `api`, `sqlserver`, `rabbitmq`, `worker`, and `dlq-recovery-worker`.

2. Know which refactored component you are exercising

Component	What it proves during this demo
<code>RabbitMqConsumerWorkerBase</code>	Both workers declare topology, connect to SQL/RabbitMQ, expose readiness, and safely requeue unexpected handling failures.
<code>BulkNotificationCommandHandler</code>	It performs the three application attempts, persists <code>DeadLettered</code> , and rejects the final main-queue delivery.
<code>DeadLetterRecoveryDeliveryHandler</code>	It accepts only a rejected main-queue command, persists <code>RecoveryPending</code> , clears the exhausted retry header, publishes to recovery, then ACKs the DLQ.
<code>DeadLetterRecoveryMessageInspector</code>	It protects against replay loops by enforcing rejected-origin and replay-limit checks; non-replayable messages go to the parking queue.

Expected readiness logs: the primary worker announces that it consumes `nsa.notifications.bulk.v1`; the recovery worker announces that it consumes `nsa.notifications.bulk.dlq` and replays through `nsa.notifications.bulk.recovery`.

3. Create a known DLQ entry

Stop only the recovery worker first. This holds the rejected command in the DLQ long enough to observe the durable `DeadLettered` state before it is replayed.

```
docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml stop dlq-recovery-worker

$env:WEEK3_FAILURE_INJECTION_SUBJECT = '[week3-poison]'
docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml up --detach --force-recreate worker
```

Submit a two-recipient command. Use a unique email pair if you will demonstrate this repeatedly.

```
$body = @{
  notifications = @(
    @{ recipientEmail = 'demo-admin@example.test'; channel = 4
      subject = '[week3-poison]'; body = 'Administrator recovery demo.' }
    @{ recipientEmail = 'demo-visitor@example.test'; channel = 4
      subject = '[week3-poison]'; body = 'Visitor recovery demo.' }
  )
} | ConvertTo-Json -Depth 4

$job = Invoke-RestMethod -Method Post `
  -Uri 'http://localhost:8080/api/v2/notifications/bulk' `
  -ContentType 'application/json' -Body $body
$job
```

Poll the returned job id until the status is `DeadLettered`.

```
Invoke-RestMethod `
  -Uri "http://localhost:8080/api/v2/notifications/bulk/${$job.jobId}"
```

Expected state before recovery starts	Why it matters
Status = DeadLettered	The primary handler persisted the terminal attempt before rejecting to the broker DLQ.
ProcessedCount = 0 ; SucceededCount = 0	The opt-in injector fails before any recipient work begins, making the path deterministic.
One command is waiting in nsa.notifications.bulk.dlq	The stopped recovery worker ensures the message is visible and cannot be immediately replayed.

4. Requeue and verify both recipients

Clear the test hook and recreate the primary worker. Start the recovery worker. The DLQ handler changes the SQL job to `RecoveryPending` , publish-confirms a message to the recovery queue, and the queue TTL sends it back to the main queue with a fresh retry budget.

```
Remove-Item Env:WEEK3_FAILURE_INJECTION_SUBJECT -ErrorAction SilentlyContinue
docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml up --detach --force-recreate `
  worker dlq-recovery-worker

# Poll until Completed (the recovery queue adds a short delay).
Invoke-RestMethod `
  -Uri "http://localhost:8080/api/v2/notifications/bulk/${$job.jobId}"
```

Then verify that both recipients received exactly one persisted notification.

```
Invoke-RestMethod `
  -Uri 'http://localhost:8080/api/v2/notifications?recipientEmail=demo-admin%40example.test'
Invoke-RestMethod `
  -Uri 'http://localhost:8080/api/v2/notifications?recipientEmail=demo-visitor%40example.test'

docker compose --env-file notification-api/.env `
  -f notification-api/compose.week3.yml logs --tail 100 worker dlq-recovery-worker
```

Final proof	Meaning
Status = Completed ; ProcessedCount = 2 ; SucceededCount = 2	The requeued command became processable after the SQL recovery transition.
One admin notification and one visitor notification	The item-level progress model prevents duplicate work on a replay.
Recovery log says “replayed ... replay 1 of 3”	The DLQ handler cleared <code>x-retry-count</code> , incremented <code>x-dlq-replay-count</code> , and used the durable recovery route.
Compose services are healthy and worker injection is blank	The environment is restored to normal operation after the demonstration.

If the message is not eligible: do not reject it from the DLQ. The recovery handler publish-confirms it to `nsa.notifications.bulk.parking` with `x-dlq-recovery-reason` , preserving it for inspection without a loss or an infinite loop.

Latest validation: after the refactor, this exact two-recipient path was exercised successfully: `DeadLettered` → `RecoveryPending` → `Completed` , with one persisted admin and one visitor notification.